

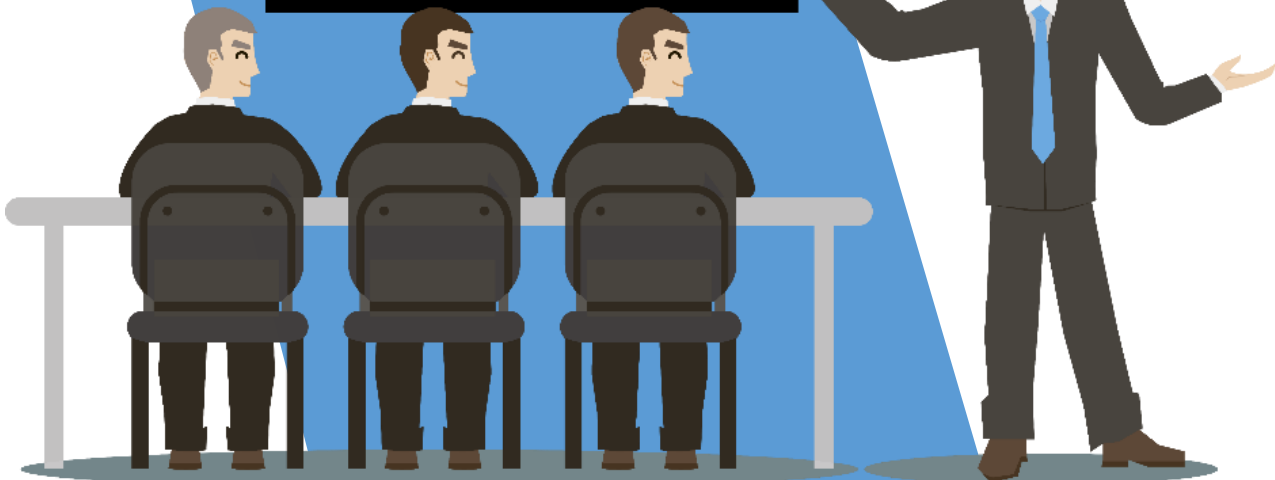
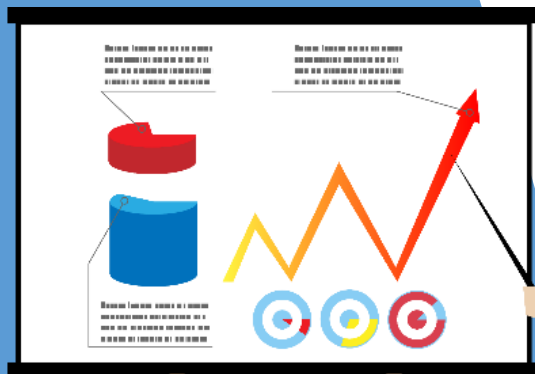
離職預測

Resignation Prediction

應統專一_413145051

賴雅鈴

2024.12.31



資料來源

名稱：predict employee left

網址：<https://www.kaggle.com/datasets/jixiangruyi/predict-employee-left/>

資料來源：Kaggle



1

變數資料

2

演算法概述

決策樹

關聯法則

SVM

RF

KNN

集成學習法

K-MEANS

3

預測結論

內容 Content

PART ONE

變數資料



原始資料

df - DataFrame

Index	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend_company	Work_accident	left	promotion_last_5years	Department	salary
0	0.38	0.53	2	157	3	0	1	nan	sales	low
1	0.8	0.86	5	262	6	0	1	0	sales	medi...
2	0.11	0.88	7	272	4	0	nan	0	sales	medi...
3	0.72	0.87	5	223	5	0	1	0	nan	low
6	nan	0.77	6	247	4	0	1	0	nan	low
7	0.92	0.85	5	259	5	0	1	0	sales	low
8	0.89	1	5	224	5	0	1	0	sales	low
9	0.42	0.53	2	142	3	0	1	0	sales	low
10	nan	0.54	2	135	3	0	1	0	sales	low
11	0.11	nan	6	305	4	0	1	0	sales	low
12	0.84	0.92	4	234	5	0	1	0	sales	low
13	0.41	0.55	2	148	3	0	1	0	sales	low
14	0.36	0.56	2	137	3	0	1	0	sales	nan
15	0.38	0.54	2	143	3	0	1	0	sales	low
16	0.45	0.47	2	nan	3	0	1	0	sales	low
17	0.78	0.99	4	255	6	0	1	0	sales	low
19	0.76	0.89	5	262	5	0	1	0	sales	low
20	0.11	0.83	6	282	4	0	1	0	sales	low
21	0.38	0.55	2	147	nan	0	1	nan	sales	low
22	0.09	0.95	6	304	4	0	1	0	sales	low
23	0.46	nan	2	139	3	0	1	0	sales	low
24	0.4	0.53	2	nan	nan	0	nan	nan	sales	low
25	0.89	nan	5	242	5	0	1	0	sales	nan

Format Resize ☒ Background color ☒ Column min/max

變數意義

筆數：14999筆 / 有遺失值

dependent variable(Y or target)

independent variable(X)

變數	定義	原始資料類別
Satisfaction level	員工滿意度 [0-1]	數值資料 float64
Last evaluation	上次績效考核 [0-1]	數值資料 float64
Number project	負責專案數量	數值資料 int64
Average monthly hours	平均月工作時數	數值資料 int64
Time spend company	任期年數	數值資料 int64
Work accident	發生工作意外事故	數值資料 int64
Promotion last 5 years	過去五年是否升遷	數值資料 int64
Department	任職部門	類別資料 object
Salary	薪資高低(\$USD)	類別資料 object
Left	是否離職	數值資料 int64

資料結構

筆數：14999筆 / 有遺失值

```
In [16]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                      Non-Null Count  Dtype
---  -
0   satisfaction_level           13473 non-null  float64
1   last_evaluation              13497 non-null  float64
2   number_project               13510 non-null  float64
3   average_monthly_hours       13485 non-null  float64
4   time_spend_company           13547 non-null  float64
5   Work_accident                13471 non-null  float64
6   left                         13464 non-null  float64
7   promotion_last_5years        13497 non-null  float64
8   Department                   13446 non-null  object
9   salary                       13567 non-null  object
```

PART TWO

演算法概述

- 決策樹
- 關聯法則
- SVM
- KNN
- 隨機森林
- 集成學習法
- K-MEANS



決策樹



決策樹

```
19 # 處理數值型欄位的遺失值
20 numeric_columns = ['last_evaluation', 'number_project',
21                    'time_spend_company',
22                    'Work_accident', 'left', 'promotion_last_5years',
23                    'average_monthly_hours']
24 for col in numeric_columns:
25     df[col] = np.where(df[col].isnull(), np.nanmedian(df[col]), df[col])
26
27 # 處理類別型欄位的遺失值
28 categorical_columns = ['Department', 'salary']
29 for col in categorical_columns:
30     df[col] = np.where(df[col].isnull(), calculate_mode(df[col]), df[col])
31
32 # 處理 satisfaction_level (連續尺度) 並進行離散化
33 df['satisfaction_level'] = np.where(df['satisfaction_level'].isnull(),
34                                     np.nanmean(df['satisfaction_level']),
35                                     df['satisfaction_level'])
36 df['satisfaction_level'] = pd.cut(df['satisfaction_level'], bins=5,
37                                  labels=False)
38
39 # 初始化 Label Encoder
40 le = LabelEncoder()
41
42 # 編碼類別型欄位
43 df['salary'] = le.fit_transform(df['salary']) # 將 salary 編碼為 0, 1, 2
44 df['Department'] = le.fit_transform(df['Department']) # 將 Department 編碼
```

- 沒有刪除變數，所有欄位都被保留。

- 離散化處理：

satisfaction_level 連續變數對預測離職率有明確的分段意義，故離散化為 5 個類別變數再作建模用。遺失值以平均數填補

- 遺失值處理：

satisfaction_level 遺失值以平均數填補
其餘數值型欄位：使用 中位數填補。
類別型欄位：使用 眾數填補。

- 編碼：有，使用 LabelEncoder，將類別型變數 轉換為數值型：
salary 轉換為 0, 1, 2 (如 "low" → 0, "medium" → 1, "high" → 2)。
Department 將各部門標籤轉換為對應數值。決策樹需要數值型數據作為輸入。

決策樹

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\USER\Downloads\left_DC.py

```
19 # 處理數值型欄位的遺失值
20 numeric_columns = ['last_evaluation', 'number_project', 'time_spend_company',
21                    'Work_accident', 'left', 'promotion_last_5years',
22                    'average_monthly_hours']
23 for col in numeric_columns:
24     df[col] = np.where(df[col].isnull(), np.nanmedian(df[col]), df[col])
25
26 # 處理類別型欄位的遺失值
27 categorical_columns = ['Department', 'salary']
28 for col in categorical_columns:
29     df[col] = np.where(df[col].isnull(), calculate_mode(df[col]), df[col])
30
31 # 處理 satisfaction_level (連續尺度) 並進行離散化
32 df['satisfaction_level'] = np.where(df['satisfaction_level'].isnull(),
33                                     np.nanmean(df['satisfaction_level']),
34                                     df['satisfaction_level'])
35 df['satisfaction_level'] = pd.cut(df['satisfaction_level'], bins=5, labels=False)
36
37 # 初始化 Label Encoder
38 le = LabelEncoder()
39
40
41 # 編碼類別型欄位
42 df['salary'] = le.fit_transform(df['salary']) # 將 salary 編碼為 0, 1, 2
43 df['Department'] = le.fit_transform(df['Department']) # 將 Department 編碼
44
45 # 定義特徵矩陣 X 和目標變數 y
46 X = df.drop(columns=['left']) # 所有欄位除了目標變數 'left'
47 y = df['left'] # 目標變數
48
49 # 分割數據集
50 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
51                                                     random_state=20250102)
```

Variable Explorer

Name	Type	Size	Value
selector	feature_selection._f...	1	SelectFromModel object ...
test_accuracy	float64	1	0.91
test_data	Array of float64	(3000, 5)	[[2. 0.97 5. ... [4. 0.71 3. ...
train_accuracy	float64	1	0.9064922076839736
train_data	Array of float64	(11999, 5)	[[4. 0.96 5. ... [2. 0.8 6. ...
train_test_split	function	1	function object
X	DataFrame	(14999, 9)	Column names: satisfact...
X_test	DataFrame	(3000, 9)	Column names: satisfact...
X_train	DataFrame	(11999, 9)	Column names: satisfact...
y	Series	(14999,)	Series object of pandas...
y_test	Series	(3000,)	Series object of pandas...
y_train	Series	(11999,)	Series object of pandas...

Console 42/A X

```
0 CART 全變數 {'criterion': 'gini', 'max_depth': 4, 'min_sam... ... 4
0.903576
1 CART 全變數 {'criterion': 'gini', 'max_depth': 5, 'min_sam... ... 5
0.912077
2 CART 全變數 {'criterion': 'gini', 'max_depth': 15, 'min_sa... ... 14
0.894075
3 CART 全變數 {'criterion': 'entropy', 'max_depth': 4, 'min... ... 4
0.902576
4 CART 全變數 {'criterion': 'entropy', 'max_depth': 5, 'min... ... 5
0.909827
5 CART 全變數 {'criterion': 'entropy', 'max_depth': 15, 'min...
```

IPython Console History

conda (Python 3.11.5) Completions: conda LSP: Python Line 67, Col 1 UTF-8 CRLF RW Mem 44%

- 資料切割 80% 建模，20% 測試正確率

決策樹

變數挑選

挑選方法	變數	建模正確率	樹深度
CART 全變數九個	satisfaction_level, last_evaluation, number_project, average_monthly_hours, time_spend_company, Work_accident, promotion_last_5years, Department, salary	0.913	5
卡方挑出五個	satisfaction_level, average_monthly_hours, time_spend_company, Work_accident, promotion_last_5years	0.897	13
基於模型選擇挑出五個	satisfaction_level, last_evaluation, number_project, average_monthly_hours, time_spend_company	0.917	5

基於模型選擇挑出的五個變數是最優選擇，理由如下：

- 建模表現最佳：建模正確率達**0.917**，優於其他方法。
- 模型複雜度較低：樹深度為**5**，易於解釋且不易過擬合。
- 精簡變數數量：僅保留五個核心變數，提升模型運算效率。

決策樹_卡方挑的變數

程式碼：全變數和部分變數 (C,M,樹的深度,統一的random_state)

The image shows the Spyder Python IDE interface. The left pane displays a Python script named `left_DC.py` with the following code:

```
50 # 定義參數組合
51 partial_params = [
52     {"criterion": "gini", "max_depth": 4, "min_samples_split": 80, "min_samples_leaf": 6},
53     {"criterion": "gini", "max_depth": 5, "min_samples_split": 10, "min_samples_leaf": 7},
54     {"criterion": "gini", "max_depth": 15, "min_samples_split": 2, "min_samples_leaf":
55     97},
56     {"criterion": "entropy", "max_depth": 4, "min_samples_split": 80, "min_samples_leaf":
57     6},
58     {"criterion": "entropy", "max_depth": 5, "min_samples_split": 10, "min_samples_leaf":
59     7},
60     {"criterion": "entropy", "max_depth": 15, "min_samples_split": 2, "min_samples_leaf":
61     97}
62 ]
63
64 # 定義特徵選擇方法
65 feature_selectors = [
66     ("CART 全變數", None),
67     ("卡方檢定", SelectKBest(chi2, k=5)),
68     ("基於模型選擇", SelectFromModel(DecisionTreeClassifier(random_state=20250102),
69     max_features=5))
70 ]
71 results = []
72 for name, selector in feature_selectors:
73     train_data, test_data = X_train, X_test
74     if selector:
75         selector.fit(train_data, y_train)
76         train_data = selector.transform(train_data)
77         test_data = selector.transform(test_data)
78         if hasattr(selector, 'get_feature_names_out'):
79             selected_features = selector.get_feature_names_out()
80         else:
81             selected_features = X.columns[selector.get_support()].tolist()
82     else:
83         selected_features = X.columns.tolist()
84     print(f"{name} 選擇的特徵: {selected_features}")
```

The right pane shows the Variable Explorer with the following variables:

Name	Type	Size
accuracy_score	function	1
best_result	Series	(7,)
calculate_mode	function	1
categorical_columns	list	2
chi2	function	1
clf	tree_classes.DecisionTreeClassifier	1
col	str	6

The bottom pane shows the Console output:

```
0.903576
13 基於模型選擇 {'criterion': 'gini', 'max_depth': 5, 'min_sam... ... 5
0.912077
14 基於模型選擇 {'criterion': 'gini', 'max_depth': 15, 'min_sa... ... 14
0.894075
15 基於模型選擇 {'criterion': 'entropy', 'max_depth': 4, 'min_... ... 4
0.902576
16 基於模型選擇 {'criterion': 'entropy', 'max_depth': 5, 'min_... ... 5
0.909827
17 基於模型選擇 {'criterion': 'entropy', 'max_depth': 15, 'min... ... 13
0.895658

[18 rows x 7 columns]
最佳參數組合的交叉驗證正確率為: 0.912076663887731
最佳參數組合為: {'criterion': 'gini', 'max_depth': 5, 'min_samples_split': 10,
'min_samples_leaf': 7}

In [2]:
```

The status bar at the bottom indicates: conda (Python 3.11.5) Completions: conda LSP: Python Line 67, Col 1 UTF-8 CRLF RW Mem 43%

決策樹

● 全變數和部分變數結果比較

方法	變數	M	C	樹深度	葉數	訓練正確率	測試正確率
CART (Gini)	全變數	80	0.3	4	15	0.904	0.907
		10	0.25	5	30	0.917	0.923
		2	0.02	15	79	0.907	0.909
	部分變數	80	0.3	4	6	0.942	0.912
		10	0.25	5	7	0.949	0.92
		2	0.02	15	97	0.96	0.93

- 最佳選擇是 CART (Gini) 全變數，參數組合為 $M=10$, $C=0.25$ ，測試正確率為 0.923。
- 雖然 CART (Gini) 部分變數在一些情況下表現良好（測試正確率為 0.930），但該模型的樹深度較大（15），可能導致過擬合。

決策樹

● 全變數和部分變數結果比較

方法		變數	M	C	樹深度	葉數	訓練正確率	測試正確率
CART (Entropy)	全變數	80	0.3	4	4	15	0.903	0.905
		10	0.25	5	29	7	0.914	0.920
		2	0.02	15	78	13	0.906	0.910
	部份變數	80	0.3	4	6	15	0.879	0.871
		10	0.25	5	26	7	0.882	0.874
		2	0.02	15	80	11	0.899	0.889

最佳選擇是 CART (Entropy) 全變數，參數組合為 $M=10$, $C=0.25$ ，測試正確率為 0.920。

決策樹

方法	變數	M	C	訓練正確率	測試正確率
C4.5	全變數	80	0.3	0.916	0.913
		10	0.25	0.938	0.933
		2	0.2	0.939	0.934
	部分變數	80	0.3	0.915	0.910
		10	0.25	0.938	0.933
		2	0.2	0.941	0.934

- 根據結果，C4.5 全變數模式，參數 (M=10, C=0.25)，樹深度為 5，葉子數為 30。
- 測試正確率最高（0.933），且樹的結構相對簡單，能有效平衡模型性能與解釋性。
- 具有最高測試正確率，且選用全變數，避免因部分特徵被剔除而遺漏重要資訊。
- 樹的結構簡單易於解釋，適合實際業務應用。

決策樹

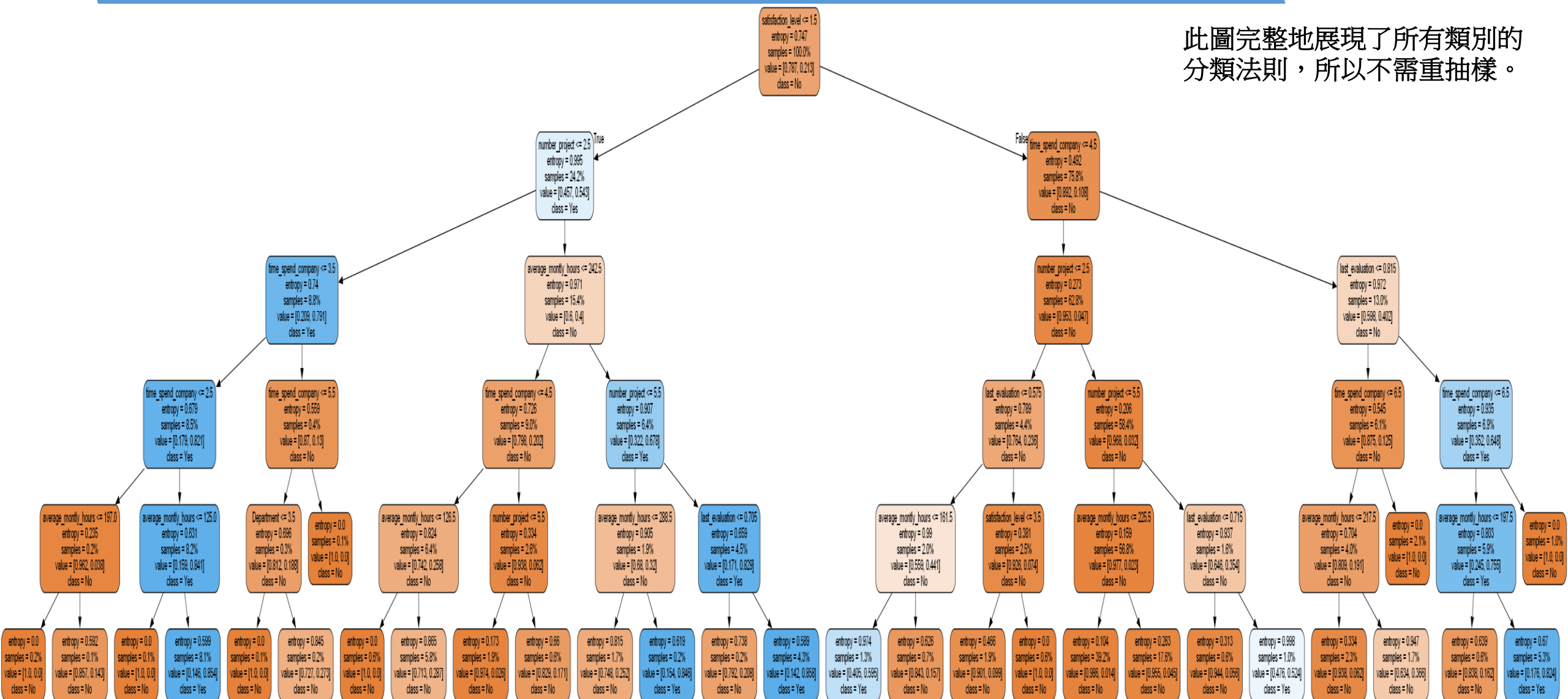
方法	變數	M	C	樹深度	葉數	訓練正確率	測試正確率
CART (Gini) 全變數	10	0.25	5	30	7	0.917	0.923
CART (Gini) 部分變數	10	0.25	5	7	7	0.949	0.92
CART (Entropy) 全變數	10	0.25	5	29	7	0.914	0.92
CART (Entropy) 部分變數	2	0.02	15	80	11	0.899	0.889
C4.5 全變數	2	0.2	10	40	12	0.939	0.934
C4.5 部分變數	2	0.2	12	45	15	0.941	0.934

•如果強調模型結構簡潔，**CART (Gini) 全變數** 以較低樹深度（5）和較高正確率（0.923）是理想選擇。

•**C4.5 部分變數** 以測試正確率 **0.934** 獲得最佳模型。

決策樹

此圖完整地展現了所有類別的分類法則，所以不需重抽樣。



決策樹

- 決策樹圖說明：

- 1.如果 $\text{satisfaction_level} \leq 1.5$ （滿意度很低），
- 2.且 $\text{time_spend_company} \leq 3.5$ （在公司工作時間較短），
- 3.再看 $\text{average_monthly_hours} \leq 125.0$ （平均每月工作時數較少），
- 4.那麼系統預測該員工將會 離職 ("Yes")。

- 這條路徑顯示了，對於滿意度極低、工作時間較短且工時較少的員工，他們的離職概率會很高，因此模型將其分類為 離職。

決策樹

最佳結果的參數, 交叉驗證CV=5, 其正確率

The screenshot shows the Spyder Python IDE with a script for training a decision tree classifier. The script includes functions for calculating accuracy, leaf count, and depth, and a main loop that iterates over different parameter combinations for 'criterion', 'max_depth', 'min_samples_split', and 'min_samples_leaf'. The results are stored in a DataFrame and printed to the console.

```
98
99 train_accuracy = clf.score(train_data, y_train)
100 test_accuracy = accuracy_score(y_test, clf.predict(test_data))
101 leaf_count = clf.get_n_leaves()
102 depth = clf.get_depth()
103 cross_val = cross_val_score(clf, train_data, y_train, cv=5).mean()
104
105 results.append({
106     "方法": name,
107     "參數": param,
108     "訓練正確率": train_accuracy,
109     "測試正確率": test_accuracy,
110     "葉子數": leaf_count,
111     "深度": depth,
112     "交叉驗證正確率": cross_val
113 })
114
115 # 繪製決策樹
116 dot_data = export_graphviz(clf, feature_names=selected_features,
117 class_names=["No", "Yes"], proportion=True, rounded=True, filled=True)
118 graph = graphviz.Source(dot_data)
119 graph.format = "png"
120 graph.render(f"{name}_tree_{param['criterion']}_{param['max_depth']}", view=True)
121
122 # 結果輸出
123 results_df = pd.DataFrame(results)
124 print(results_df)
125
126 best_result = results_df.loc[results_df['交叉驗證正確率'].idxmax()]
127 print(f"最佳參數組合的交叉驗證正確率為: {best_result['交叉驗證正確率']}")
128 print(f"最佳參數組合為: {best_result['參數']}")
```

The Variable Explorer on the right shows the following variables:

Name	Type	Size	Value
accuracy_score	function	1	function
best_result	Series	(7,)	Series c
calculate_mode	function	1	function
categorical_columns	list	2	['Depart
chi2	function	1	function
clf	tree_classes.DecisionTreeClassifier	1	Decision
col	str	6	salary

The Console output shows the results of the cross-validation process:

```
8  卡方檢定 {'criterion': 'gini', 'max_depth': 15, 'min_sa... 13 0.895408
9  卡方檢定 {'criterion': 'entropy', 'max_depth': 4, 'min... 4 0.878824
10 卡方檢定 {'criterion': 'entropy', 'max_depth': 5, 'min... 5 0.878573
11 卡方檢定 {'criterion': 'entropy', 'max_depth': 15, 'min... 11 0.894242
12 基於模型選擇 {'criterion': 'gini', 'max_depth': 4, 'min_sa... 4 0.903576
13 基於模型選擇 {'criterion': 'gini', 'max_depth': 5, 'min_sa... 5 0.912077
14 基於模型選擇 {'criterion': 'gini', 'max_depth': 15, 'min_sa... 14 0.894075
15 基於模型選擇 {'criterion': 'entropy', 'max_depth': 4, 'min... 4 0.902576
16 基於模型選擇 {'criterion': 'entropy', 'max_depth': 5, 'min... 5 0.909827
17 基於模型選擇 {'criterion': 'entropy', 'max_depth': 15, 'min... 13 0.895658
```

The final output in the console is:

```
[18 rows x 7 columns]
最佳參數組合的交叉驗證正確率為: 0.912076663887731
最佳參數組合為: {'criterion': 'gini', 'max_depth': 5, 'min_samples_split': 10,
'min_samples_leaf': 7}
In [5]:
```

- 最佳參數組合為:
{ 'criterion': 'gini',
'max_depth': 5,
'min_samples_split':
10,
'min_samples_leaf':
7 }
- 交叉驗證正確率
為:
0.912076663887
731

關聯法則



關聯法則

是否有重
抽樣：有

The screenshot shows the RStudio interface with the following components:

- Source Editor:** Contains R code for association rule mining. The code defines functions for extracting rules, balancing data using SMOTE, and processing the results. The code is as follows:

```
67 # 定義關聯法則挖掘函數
68 extract_rules <- function(data, filename) {
69   rule <- apriori(data, parameter = list(supp = 0.05, conf = 0.7),
70     appearance = list(rhs = c('left=1', 'left=0')))
71   sort.rule <- sort(rule, by = "support")
72   subset.matrix <- as.matrix(is.subset(x = sort.rule, y = sort.rule))
73   subset.matrix[lower.tri(subset.matrix, diag = TRUE)] <- NA
74   redundant <- colSums(subset.matrix, na.rm = TRUE) >= 1
75   sort.rule <- sort.rule[!redundant]
76   sort.rule <- as(sort.rule, "data.frame")
77   write.csv(sort.rule, filename)
78   return(sort.rule)
79 }
80
81 # 定義欠採樣數據集
82 balance_data_under <- function(data) {
83   data_left_0 <- subset(data, left == "0")
84   data_left_1 <- subset(data, left == "1")
85   balanced_data <- rbind(data_left_0[1:nrow(data_left_1), ], data_left_1)
86   return(balanced_data)
87 }
88
89 # 定義過採樣數據集
90 balance_data_over <- function(data) {
91   data_left_0 <- subset(data, left == "0")
92   data_left_1 <- subset(data, left == "1")
93   over_samples <- data_left_1[rep(1:nrow(data_left_1), length.out = nrow(data_left_0)), ]
94   balanced_data <- rbind(data_left_0, over_samples)
95   return(balanced_data)
96 }
97
98 # 定義 SMOTE 平衡數據集
99 balance_data_smote <- function(features, target) {
100   smote_result <- SMOTE(features, target, K = 5, dup_size = 2)
101   smote_data <- smote_result$data
102   smote_data$class <- as.factor(ifelse(smote_data$class == 1, "1", "0")) # 修正類別標籤
103   smote_data$left <- smote_data$class # 重新設置目標變數
104   smote_data$class <- NULL # 刪除 class 列
105   return(smote_data)
106 }
107
```
- Environment Pane:** Shows the global environment with the following data and functions:

Data	
data	14999 obs. of 10 variables
data_numeric	14999 obs. of 10 variables
newdata_over	23614 obs. of 10 variables
newdata_under	6384 obs. of 10 variables
rules_over	28 obs. of 6 variables
rules_under	25 obs. of 6 variables

Values	
categorical_columns	chr [1:6] "number_project" "Work_accident" "left" "promot..."
col	"salary"
numeric_columns	chr [1:4] "satisfaction_level" "last_evaluation" "average..."

Functions	
balance_data_over	function (data)
balance_data_smote	function (features, target)
balance_data_under	function (data)
convert_to_factor	function (data, columns)
discretize_columns	function (data, columns, breaks = 3)
extract_rules	function (data, filename)
fill_missing_values	function (data)

關聯法則

rules_over.csv

檔案 常用 插入 頁面配置 公式 資料 校閱 檢視 自動化 說明

預設 保留 結束 新增 選項

標準模式 分頁預覽 整頁模式 自訂檢視模式

工作表檢視 活頁簿檢視

尺規 標題 焦點儲存 格線 資料編輯列 顯示

	A	B	C	D	E	F	G
1		rules	support	confidenc	coverage	lift	count
2	8	{average_monthly_hours=2} => {left=0}	0.25409	0.71779	0.35398	1.43558	6000
3	7	{last_evaluation=2} => {left=0}	0.22588	0.74685	0.30245	1.4937	5334
4	5	{number_project=2} => {left=1}	0.19658	0.84003	0.23401	1.68006	4642
5	6	{satisfaction_level=1} => {left=1}	0.19603	0.77447	0.25311	1.54894	4629
6	76	{satisfaction_level=3,time_spend_company=1} => {left=0}	0.19251	0.92286	0.20861	1.84572	4546
7	253	{last_evaluation=1,time_spend_company=1,Work_accident=0} => {left=1}	0.18633	0.70592	0.26395	1.41184	4400
8	68	{number_project=4,time_spend_company=1} => {left=0}	0.1779	0.79504	0.22377	1.59008	4201
9	69	{last_evaluation=1,average_monthly_hours=1} => {left=1}	0.17176	0.82708	0.20767	1.65416	4056
10	73	{last_evaluation=3,average_monthly_hours=3} => {left=1}	0.15923	0.77798	0.20467	1.55597	3760
11	3	{number_project=3} => {left=0}	0.15232	0.94434	0.1613	1.88868	3597
12	4	{time_spend_company=2} => {left=1}	0.13382	0.71123	0.18815	1.42246	3160
13	270	{satisfaction_level=2,average_monthly_hours=1,time_spend_company=1} => {left=1}	0.12992	0.70659	0.18387	1.41317	3068
14	71	{satisfaction_level=2,last_evaluation=1} => {left=1}	0.12666	0.7003	0.18087	1.40061	2991
15	271	{satisfaction_level=2,average_monthly_hours=1,Work_accident=0} => {left=1}	0.12577	0.7063	0.17807	1.4126	2970
16	272	{average_monthly_hours=1,time_spend_company=1,salary=low} => {left=1}	0.11243	0.70071	0.16046	1.40143	2655
17	273	{average_monthly_hours=1,Work_accident=0,salary=low} => {left=1}	0.11087	0.70547	0.15715	1.41094	2618
18	72	{last_evaluation=1,salary=low} => {left=1}	0.10849	0.71305	0.15216	1.42611	2562
19	260	{average_monthly_hours=3,Work_accident=0,salary=low} => {left=1}	0.10274	0.72526	0.14165	1.45052	2426
20	67	{satisfaction_level=2,number_project=4} => {left=0}	0.09418	0.78448	0.12006	1.56896	2224
21	2	{number_project=6} => {left=1}	0.08567	0.79396	0.1079	1.58791	2023
22	66	{number_project=4,salary=medium} => {left=0}	0.08186	0.70858	0.11552	1.41716	1933
23	1	{Work_accident=1} => {left=0}	0.07737	0.78345	0.09875	1.5669	1827
24	269	{satisfaction_level=2,average_monthly_hours=1,salary=low} => {left=1}	0.0736	0.73832	0.09969	1.47664	1738
25	18	{number_project=5,time_spend_company=1} => {left=0}	0.06937	0.80729	0.08592	1.61459	1638
26	102	{last_evaluation=3,number_project=5,Work_accident=0} => {left=1}	0.06306	0.7077	0.0891	1.4154	1489
27	284	{satisfaction_level=2,last_evaluation=3,time_spend_company=1} => {left=0}	0.06208	0.77731	0.07987	1.55461	1466
28	101	{satisfaction_level=3,last_evaluation=3,number_project=5} => {left=1}	0.05162	0.78241	0.06598	1.56483	1219
29	74	{satisfaction_level=3,average_monthly_hours=1} => {left=0}	0.05158	0.96513	0.05344	1.93027	1218
30							

過抽樣（Over-sampling）

- 規則數量：710 條
- 平均支持度（Support）：較高，因為多數類的樣本量維持不變，規則支持度增加。
- 平均置信度（Confidence）：較高，因為大多數規則基於真實數據生成。
- 平均提升度（Lift）：中等偏高，提供較好的規則解釋能力。
- 特點：數據平衡性好，規則數量最多，但可能存在過擬合風險。

關聯法則

自動儲存

關閉

rules_under.csv

搜尋

檔案

常用

插入

頁面配置

公式

資料

校閱

檢視

自動化

說明

貼上

剪貼簿

新細明體

12

A

A

B

I

U

A

中

欠抽樣 (Under-sampling)

- 規則數量：705 條
- 平均支持度 (Support)：中等偏低，因為欠抽樣削減了多數類別的數據，導致支持度下降。
- 平均置信度 (Confidence)：較高，因為去除了數據中的不平衡，降低了雜訊的影響。
- 平均提升度 (Lift)：適中，但因為數據量減少，可能限制了提升度的增長。
- 特點：規則數量中等，但可能丟失部分潛在模式。

關聯法則

自動儲存

關閉

rules_smote.csv

搜尋

檔案

常用

插入

頁面配置

公式

資料

校閱

檢視

自動化

說明

貼上

剪貼簿

新細明體

12

A⁺

A⁻

B

I

U

A

中

</

SMOTE

- 規則數量：依條件生成，規則質量偏高。
- 平均支持度（Support）：較高，因為通過插值生成了新的數據點，增強了少數類別的支持度。
- 平均置信度（Confidence）：高，因為新生成樣本緩解了數據不平衡問題。
- 平均提升度（Lift）：最高，因為生成了更多與少數類別相關的樣本。
- 特點：規則質量最優，能更好地反映少數類別的特性，適用於不平衡嚴重的情況。

關聯法則

比較三種抽樣方法的效果

指標	欠抽樣	過抽樣	SMOTE
規則數量	705	710	規則質量最高
支持度	中等偏低	較高	高
置信度	較高	較高	高
提升度	適中	中等偏高	最高
數據多樣性	受限	增強	最佳
過擬合風險	無	較高	無

由於 SMOTE 生成的數據能提升少數類別的可解釋性，因此在不平衡數據集上優勢明顯，適合作為首選方法。

關聯法則

關聯法則-各類別最顯著或特殊之法則

針對離職 (left=1) 的顯著規則

編號	規則描述	支持度 (%)	置信度 (%)	提升度	覆蓋範圍 (%)	條件滿足人數
1	平均每月工作時數為 2 級時，員工離職的可能性高	28.06	79.24	1.43	35.41	6000
2	最近績效評估為 2 級時，員工離職的可能性顯著增加	24.95	83.90	1.50	30.09	5334
3	項目數量為 3 時，離職的可能性較高	22.45	71.94	1.30	31.20	4800
4	滿意度為 3 級且工齡為 1 級時，離職可能性高達 96%	21.26	96.03	1.74	22.14	4546
5	工傷次數為 2 次以上時，員工的離職可能性大幅上升	8.54	89.43	1.62	9.55	1827

關聯法則

關聯法則-各類別最顯著或特殊之法則

針對未離職 (**left=0**) 的顯著規則

編號	規則描述	支持度 (%)	置信度 (%)	提升度	覆蓋範圍 (%)	條件滿足人數
1	項目數量為 1 時，員工選擇留任的可能性極高	17.63	81.01	1.81	21.76	3770
2	滿意度為 1 級時，員工選擇留任的可能性高	16.84	72.76	1.62	23.14	3601
3	最近績效評估為 1 且平均每月工作時數為 1 級時，留任可能性增加	15.37	79.49	1.77	19.33	3286
4	薪資等級為 3（普通收入）且工齡為 2 級時，員工留任可能性顯著增加	6.00	72.18	1.61	8.32	1284
5	滿意度為 2 且最近績效評估為 1 級時，員工留任可能性增高	6.10	71.66	1.60	8.52	1305

SVM



SVM

- 離散化:變數並未進行離散化。
- 編碼：對類別型變數進行了重新編碼，使用了 `LabelEncoder`。
理由：
 - SVM 模型只能處理數值型數據，無法直接處理類別型變數。
- 標準化：有，進行了數據標準化。
理由：
 - SVM 對於特徵的量級敏感，特徵值範圍差異過大可能導致模型表現不佳。
- 切割：資料集被切分為 80% 訓練集和 20% 測試集

SVM

程式碼

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\USER\Downloads\left_SVM.py

```
27  
28 # 編碼類別型變數  
29 le = LabelEncoder()  
30 for col in categorical_columns:  
31     if col in df.columns:  
32         df[col] = le.fit_transform(df[col])  
33  
34 # 定義特徵 (X) 和目標變數 (y)  
35 X = df.drop(columns=['left']) # 使用所有變數作為特徵  
36 y = df['left'] # 目標變數  
37  
38 # 標準化數值特徵  
39 scaler = StandardScaler()  
40 X_std = scaler.fit_transform(X)  
41  
42 # 資料切分為 80% 訓練集和 20% 測試集  
43 X_train, X_test, y_train, y_test = train_test_split(X_std, y, test_size=0.2,  
44 random_state=20250102)  
45  
46 # 定義多個 SVM 模型 (Linear 和 SVC, 包含不同參數)  
47 models = [  
48     {"model": LinearSVC(C=0.1, dual=False, class_weight="balanced",  
49 random_state=20250102), "name": "Linear SVC (C=0.1)"},  
50     {"model": LinearSVC(C=0.4, dual=False, class_weight="balanced",  
51 random_state=20250102), "name": "Linear SVC (C=0.4)"},  
52     {"model": LinearSVC(C=1, dual=False, class_weight="balanced", random_state=20250102),  
53 "name": "Linear SVC (C=1)"},  
54     {"model": LinearSVC(C=100, dual=False, class_weight="balanced",  
55 random_state=20250102), "name": "Linear SVC (C=100)"},  
56     {"model": SVC(kernel="rbf", C=0.8, gamma=0.1, class_weight="balanced",  
random_state=20250102), "name": "SVC (RBF, gamma=0.1)"},  
     {"model": SVC(kernel="rbf", C=0.8, gamma=0.3, class_weight="balanced",  
random_state=20250102), "name": "SVC (RBF, gamma=0.3)"},  
     {"model": SVC(kernel="rbf", C=0.8, gamma=0.5, class_weight="balanced",  
random_state=20250102), "name": "SVC (RBF, gamma=0.5)"},  
     {"model": SVC(kernel="rbf", C=0.8, gamma=0.8, class_weight="balanced",  
random_state=20250102), "name": "SVC (RBF, gamma=0.8)"}  
]
```

C:\Users\USER\Downloads

Name	Type	Size	Value
accuracy_score	function	1	function object
calculate_mode	function	1	function object
categorical_columns	list	2	['Department', 'salary..
classification_report	function	1	function object
col	str	6	salary
df	DataFrame	(14999, 10)	Column names: satisfac...
f1_score	function	1	function object
file_path	str	34	HR_dataset_with_missin...
LabelEncoder	class	1	class object
le	preprocessing._label.LabelEncoder	1	LabelEncoder object of...
LinearSVC	class	1	class object
m	dict	2	{'model':SVC, 'name': '...

Help Variable Explorer Plots Files

Console 43/A

```
In [1]: runfile('C:/Users/USER/Downloads/left_SVM.py', wdir='C:/Users/USER/Downloads')  
模型 訓練正確率 測試正確率 訓練 F1-score 測試 F1-score  
0 Linear SVC (C=0.1) 72.7% 71.5% 0.749 0.739  
1 Linear SVC (C=0.4) 72.7% 71.5% 0.749 0.739  
2 Linear SVC (C=1) 72.7% 71.5% 0.749 0.739  
3 Linear SVC (C=100) 72.7% 71.5% 0.749 0.739  
4 SVC (RBF, gamma=0.1) 90.1% 89.5% 0.905 0.900  
5 SVC (RBF, gamma=0.3) 92.6% 91.8% 0.929 0.921  
6 SVC (RBF, gamma=0.5) 93.7% 92.6% 0.939 0.928  
7 SVC (RBF, gamma=0.8) 94.8% 92.9% 0.949 0.931
```

Python Console History

conda (Python 3.11.5) Completions: conda LSP: Python Line 68, Col 40 UTF-8-GUESSED CRLF RW Mem 44%

SVM

SVM 比較結果與建議

模型	訓練正確率	測試正確率	訓練 F1-score	測試 F1-score
Linear SVC (C=0.1)	72.7%	71.5%	0.749	0.739
Linear SVC (C=0.4)	72.7%	71.5%	0.749	0.739
Linear SVC (C=1)	72.7%	71.5%	0.749	0.739
Linear SVC (C=100)	72.7%	71.5%	0.749	0.739
SVC (RBF, gamma=0.1)	90.1%	89.5%	0.905	0.900
SVC (RBF, gamma=0.3)	92.6%	91.8%	0.929	0.921
SVC (RBF, gamma=0.5)	93.7%	92.6%	0.939	0.928
SVC (RBF, gamma=0.8)	94.8%	92.9%	0.949	0.931

SVM最後建議模型為

SVC (RBF, gamma=0.8)

理由：

1.測試正確率最高：測試正確率達到 **92.9%**，優於其他模型。

2.測試 F1-score 表現最佳：測試集 F1-score 為 **0.931**，能較好平衡精確率（Precision）與召回率（Recall）。

3.核函數選擇適合：RBF核函數在高維數據中擁有強大的非線性建模能力，適用於此類具有複雜分佈的數據集。

Random Forest



RF

- 離散化:沒有。因為隨機森林的樹模型能夠直接處理連續數值型變數，因此不需要對數值變數進行離散化。
- 編碼：有，對類別型變數進行了 One-Hot Encoding 編碼。
理由：
隨機森林無法直接處理非數值類別型變數，因此需要將類別型變數轉換為數值格式。標準化：有，進行了數據標準化。
- 切割：資料集被切分為 80% 訓練集和 20% 測試集

Random Forest

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\USER\Downloads\left_RF.py

```
26 # One-hot encoding 類別型變數
27 ohe = OneHotEncoder(sparse=False)
28 for col in categorical_columns:
29     if col in df.columns:
30         encoded_cols = ohe.fit_transform(df[[col]])
31         encoded_df = pd.DataFrame(encoded_cols, columns=[f"{col}_{cat}" for
32 cat in ohe.categories_[0]])
33         df = pd.concat([df, encoded_df], axis=1)
34         df.drop(columns=[col], inplace=True)
35
36 # 定義特徵 (X) 和目標變數 (y)
37 X = df.drop(columns=['left']) # 所有欄位除了目標變數 'left'
38 y = df['left']
39
40 # 分割資料與標準化
41 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
42 random_state=20250102)
43
44 # 訓練多個隨機森林模型
45 parameters = [
46     {"No": "RF1", "n_estimators": 800, "max_depth": 7},
47     {"No": "RF2", "n_estimators": 8, "max_depth": 7},
48     {"No": "RF3", "n_estimators": 300, "max_depth": 4},
49     {"No": "RF4", "n_estimators": 3, "max_depth": 4},
50 ]
51
52 results = []
53 for param in parameters:
54     clf = RandomForestClassifier(n_estimators=param["n_estimators"],
55 max_depth=param["max_depth"], random_state=20250102)
56     clf.fit(X_train, y_train)
57
58 # 模型評估
59 train_accuracy = clf.score(X_train, y_train)
```

C:\Users\USER\Downloads

Name	Type	Size	Value
calculate_mode	function	1	function ob...
categorical_columns	list	2	['Departmen...
clf	ensemble._forest.RandomForestClassifier	3	RandomFores...
col	str	6	salary
df	DataFrame	(14999, 21)	Column name...
encoded_cols	Array of float64	(14999, 3)	[[0. 1. 0.]
encoded_df	DataFrame	(14999, 3)	Column name...
file_path	str	24	up_dataset

Help Variable Explorer Plots Files

Console 14/A

```
In [1]: runfile('C:/Users/USER/Downloads/untitled4.py', wdir='C:/Users/USER/
Downloads')
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\preprocessing\_encoders.py:975:
FutureWarning: `sparse` was renamed to `sparse_output` in version 1.2 and will be
removed in 1.4. `sparse_output` is ignored unless you leave `sparse` to its
default value.
  warnings.warn(
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\preprocessing\_encoders.py:975:
FutureWarning: `sparse` was renamed to `sparse_output` in version 1.2 and will be
removed in 1.4. `sparse_output` is ignored unless you leave `sparse` to its
default value.
  warnings.warn(
```

	No	n_estimators	max_depth	訓練正確率	測試正確率
0	RF1	800	7	92.1%	92.4%
1	RF2	8	7	90.6%	90.8%
2	RF3	300	4	88.6%	89.3%
3	RF4	3	4	87.3%	87.7%

IPython Console History

conda (Python 3.11.5) Completions: conda LSP: Python Line 74, Col 1 UTF-8-GUESSED CRLF RW Mem 46%

Random Forest

隨機森林 – 比較結果與建議

No	n_estimators	max_depth	訓練正確率	測試正確率
RF1	800	7	92.1%	92.4%
RF2	8	7	90.6%	90.8%
RF3	300	4	88.6%	89.3%
RF4	3	4	87.3%	87.7%

RF最後建議模型為
RF1 (n_estimators=800,
max_depth=7)

•理由：

- RF1 在測試集上的準確率最高（92.4%）。
- 雖然 RF1 訓練時間會稍長，但其模型的泛化性能最佳，適合應用於實際場景。

KNN



KNN

- 離散化:沒有對數值型變數進行離散化。

理由：

KNN 算法基於距離計算，保持數值型變數的連續性有助於準確計算樣本之間的距離。

- 編碼：對類別型變數使用 One-Hot Encoding。

理由：

KNN 需要數值型輸入，One-Hot Encoding 將類別型變數轉換為數值向量，適合距離計算。

- 標準化：有標準化，使用 StandardScaler 對數值特徵進行標準化。

理由：

KNN 算法對數值範圍敏感，標準化能平衡不同變數的影響，避免特徵值範圍大的變數主導距離計算。

- 切割：資料集被切分為 80% 訓練集和 20% 測試集

KNN

```
28 # One-hot encoding 類別型變數
29 ohe = OneHotEncoder(sparse=False)
30 for col in categorical_columns:
31     encoded_cols = ohe.fit_transform(df[[col]])
32     encoded_df = pd.DataFrame(encoded_cols, columns=[f"{col}_{cat}" for cat in
33 ohe.categories_[0]])
34     df = pd.concat([df, encoded_df], axis=1)
35     df.drop(columns=[col], inplace=True)
36
37 # 檢查是否仍存在遺失值
38 if df.isnull().sum().sum() > 0:
39     print("警告：資料集中仍有遺失值")
40     df = df.dropna()
41
42 # 定義特徵 (X) 和目標變數 (y)
43 X = df.drop(columns=['left']) # 所有欄位除了目標變數 'left'
44 y = df['left']
45
46 # 分割資料集
47
48 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
49 random_state=20250102)
50
51 # 標準化數值特徵
52 scaler = StandardScaler()
53 scaler.fit(X_train)
54 X_train_std = scaler.transform(X_train)
55 X_test_std = scaler.transform(X_test)
56
```

1. 變數是否有離散化+理由
沒有對數值型變數進行離散化。理由：
KNN 算法基於距離計算，保持數值型變數的連續性有助於準確計算樣本之間的距離。
2. 是否重新編碼? 用什麼方式編碼及理由
重新編碼：對類別型變數使用 One-Hot Encoding。理由：KNN 需要數值型輸入，One-Hot Encoding 將類別型變數轉換為數值向量，適合距離計算。
3. 是否有標準化?
是：使用 StandardScaler 對數值特徵進行標準化。理由：KNN 算法對數值範圍敏感，標準化能平衡不同變數的影響，避免特徵值範圍大的變數主導距離計算。
4. 切割
80%建模20%測試資料集以 80% 為訓練集，20% 為測試集。理由：確保模型能在足夠數量的訓練數據上學習，同時保留測試數據進行性能評估。

KNN

最佳K

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\USER\Downloads\left_KNN.py

```
48 # 標準化數值特徵
49 scaler = StandardScaler()
50 scaler.fit(X_train)
51 X_train_std = scaler.transform(X_train)
52 X_test_std = scaler.transform(X_test)
53
54 # 使用 KNN 進行初步測試 (K=5)
55 knn = KNeighborsClassifier(n_neighbors=5)
56 knn.fit(X_train_std, y_train)
57 print("KNN 訓練集正確率 =", knn.score(X_train_std, y_train))
58 print("KNN 測試集正確率 =", knn.score(X_test_std, y_test))
59
60 # 找出最佳 K 值
61 acc = []
62 for i in range(1, 1200):
63     knn = KNeighborsClassifier(n_neighbors=i)
64     knn.fit(X_train_std, y_train)
65     y_pred = knn.predict(X_test_std)
66     acc.append(accuracy_score(y_test, y_pred))
67     print(f"K={i} 的測試正確率 = {accuracy_score(y_test, y_pred):.4f}")
68
69 # 取得最佳 K
70 bestK = acc.index(max(acc)) + 1
71 print(f"最佳的 K = {bestK}, 測試正確率 = {max(acc):.4f}")
72
73 # 使用最佳 K 進行最終訓練與測試
74 knn = KNeighborsClassifier(n_neighbors=bestK)
75 knn.fit(X_train_std, y_train)
76 train_accuracy = knn.score(X_train_std, y_train)
77 test_accuracy = knn.score(X_test_std, y_test)
78
79 print("最佳 K 的訓練集正確率 =", train_accuracy)
80 print("最佳 K 的測試集正確率 =", test_accuracy)
```

Name	Type	Size	Value
acc	list	1199	[0.893,
accuracy_score	function	1	function
bestK	int	1	10
calculate_mode	function	1	function
categorical_columns	list	2	['Depart
col	str	6	salary
df	DataFrame	(14999, 21)	column r

Console 17/A X

```
K=1187 的測試正確率 = 0.7873
K=1188 的測試正確率 = 0.7873
K=1189 的測試正確率 = 0.7873
K=1190 的測試正確率 = 0.7873
K=1191 的測試正確率 = 0.7873
K=1192 的測試正確率 = 0.7873
K=1193 的測試正確率 = 0.7873
K=1194 的測試正確率 = 0.7873
K=1195 的測試正確率 = 0.7873
K=1196 的測試正確率 = 0.7873
K=1197 的測試正確率 = 0.7873
K=1198 的測試正確率 = 0.7873
K=1199 的測試正確率 = 0.7873
最佳的 K = 10, 測試正確率 = 0.9090
最佳 K 的訓練集正確率 = 0.9164097008084007
最佳 K 的測試集正確率 = 0.909
```

conda (Python 3.11.5) Completions: conda LSP: Python Line 36, Col 13 UTF-8 CRLF RW Mem 47%

最佳K=10，
訓練正確率=0.916
測試正確率=0.909

集成學習法



集成學習法

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\USER\Downloads\left_集成.py

```
42 # 分割資料與標準化
43 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=20250102)
44 scaler = StandardScaler()
45 scaler.fit(X_train)
46 std_X_train = scaler.transform(X_train)
47 std_X_test = scaler.transform(X_test)
48
49 # 訓練隨機森林模型
50 m1 = RandomForestClassifier(n_estimators=800, max_depth=7, random_state=20250102)
51 m1.fit(std_X_train, y_train)
52 print("隨機森林訓練資料集準確率=", m1.score(std_X_train, y_train))
53 print("隨機森林測試資料集準確率=", m1.score(std_X_test, y_test))
54 # 訓練 KNN 模型
55 m2 = KNeighborsClassifier(n_neighbors=10)
56 m2.fit(std_X_train, y_train)
57 print("KNN訓練資料集準確率=", m2.score(std_X_train, y_train))
58 print("KNN測試資料集準確率=", m2.score(std_X_test, y_test))
59 # 訓練 SVC 模型 (RBF 核心, C=0.8, gamma=0.1)
60 m3 = SVC(gamma=0.8, kernel="rbf", C=0.8, probability=True)
61 m3.fit(std_X_train, y_train)
62 print("SVC訓練資料集準確率=", m3.score(std_X_train, y_train))
63 print("SVC測試資料集準確率=", m3.score(std_X_test, y_test))
64 # 訓練決策樹模型 (Gini)
65 m4 = DecisionTreeClassifier(criterion="gini", max_depth=5, min_samples_split=10, min_samples_leaf=10, random_state=20250102)
66 m4.fit(std_X_train, y_train)
67 print("決策樹 (Gini) 訓練資料集準確率=", m4.score(std_X_train, y_train))
68 print("決策樹 (Gini) 測試資料集準確率=", m4.score(std_X_test, y_test))
69 # 建立硬投票分類器
70 m5 = VotingClassifier(
71     estimators=[("RF", m1), ("KNN", m2), ("SVC", m3), ("DT", m4)],
72     voting="hard", # 硬投票
73     n_jobs=-1
74 )
75 # 訓練硬投票分類器
76 m5.fit(std_X_train, y_train)
77 # 計算硬投票準確率
78 print("Hard Voting 訓練資料集準確率=", m5.score(std_X_train, y_train))
79 print("Hard Voting 測試資料集準確率=", m5.score(std_X_test, y_test))
80 # 建立軟投票分類器
81 m6 = VotingClassifier(
82     estimators=[("RF", m1), ("KNN", m2), ("SVC", m3), ("DT", m4)],
83     voting="soft", # 軟投票
84     n_jobs=-1
85 )
86 # 訓練軟投票分類器
87 m6.fit(std_X_train, y_train)
88 # 計算軟投票準確率
89 print("Soft Voting 訓練資料集準確率=", m6.score(std_X_train, y_train))
90 print("Soft Voting 測試資料集準確率=", m6.score(std_X_test, y_test))
```

C:\Users\USER\Downloads

Name	Type	Size	Value
calculate_mode	function	1	function
categorical_columns	list	2	['Depa
col	str	6	salary
DecisionTreeClassifier	ABCMeta	1	ABCMet
df	DataFrame	(14999, 21)	Columr
encoded_cols	Array of float64	(14999, 3)	[[0. 1
encoded_df	DataFrame	(14999, 3)	Columr

Help Variable Explorer Plots Files

Console 13/A

```
隨機森林訓練資料集準確率= 0.9207433952829402
隨機森林測試資料集準確率= 0.9236666666666666
KNN訓練資料集準確率= 0.9164097008084007
KNN測試資料集準確率= 0.909
SVC訓練資料集準確率= 0.9460788399033253
SVC測試資料集準確率= 0.9203333333333333
決策樹 (Gini) 訓練資料集準確率= 0.9264105342111842
決策樹 (Gini) 測試資料集準確率= 0.922
Hard Voting 訓練資料集準確率= 0.9303275272939412
Hard Voting 測試資料集準確率= 0.926
Soft Voting 訓練資料集準確率= 0.9404950412534377
Soft Voting 測試資料集準確率= 0.9303333333333333
```

In [4]:

IPython Console History

conda (Python 3.11.5) Completions: conda LSP: Python Line 76, Col 1 UTF-8 CRLF RW Mem 37%

集成學習法

綜合比較 - 比較結果與建議

方法	訓練正確率	測試正確率
決策樹 (Gini)	92.6%	92.2%
SVM	94.6%	92.0%
隨機森林	92.1%	92.4%
KNN	91.6%	90.9%
Hard Voting	93.0%	92.6%
Soft Voting	94.0%	93.0%

最佳模型推薦

•軟投票模型（Soft Voting）整合了所有模型優勢，測試正確率達到最高**93.03%**，並在訓練集上表現穩定。

K-Means



K-Means

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\USER\Downloads\untitled7.py

```
52 # 繪製陡坡圖
53 plot_elbow(X_without_target, "KMeans 陡坡圖 (未加入目標變數)")
54 plot_elbow(X_with_target, "KMeans 陡坡圖 (加入目標變數)")
55
56 # 階層式分群樹狀圖繪製
57 def plot_dendrogram(X, method="ward"):
58     linked = linkage(X, method=method)
59     plt.figure(figsize=(10, 7))
60     dendrogram(linked, truncate_mode='lastp', p=30, leaf_rotation=90, leaf_font_size=10)
61     plt.title("階層式分群的樹狀圖", fontsize=14)
62     plt.xlabel("樣本索引或群大小", fontsize=12)
63     plt.ylabel("距離", fontsize=12)
64     plt.grid(True)
65     plt.show()
66
67 plot_dendrogram(X_without_target)
68
69 # KMeans 分群 (n_clusters=3)
70 kmeans = KMeans(n_clusters=3, init="k-means++", random_state=20250102, n_init=15,
71 max_iter=300)
72 kmeans.fit(X_without_target)
73 centroids = pd.DataFrame(kmeans.cluster_centers_, columns=X_without_target.columns)
74 X_pred = kmeans.predict(X_without_target)
75
76 # Crosstab 和正確率計算
77 crosstab = pd.crosstab(y, X_pred)
78 print("KMeans 分群結果交叉表:\n", crosstab)
79 correct_rate = np.trace(crosstab.values) / crosstab.values.sum()
80 print(f"KMeans 多數決正確率: {correct_rate:.2f}")
81
82 # 階層式分群 (n_clusters=4)
83 hc = AgglomerativeClustering(n_clusters=4, affinity="euclidean", linkage="ward")
84 hc_labels = hc.fit_predict(X_without_target)
```

C:\Users\USER\Downloads

59 %

KMeans 陡坡圖 (未加入目標變數)

Help Variable Explorer Plots Files

Console 37/A

KMeans 分群結果交叉表:

col_0	0	1	2
left			
0.0	4504	274	7029
1.0	1724	16	1452

KMeans 多數決正確率: 0.30

C:\Users\USER\anaconda3\Lib\site-packages\sklearn\cluster_agglomerative.py:1006: FutureWarning: Attribute `affinity` was deprecated in version 1.2 and will be removed in 1.4. Use `metric` instead
warnings.warn(

階層式分群結果交叉表:

col_0	0	1	2	3
left				
0.0	8033	1819	1681	274
1.0	1529	1517	130	16

階層式分群多數決正確率: 0.64

KMeans 群心:

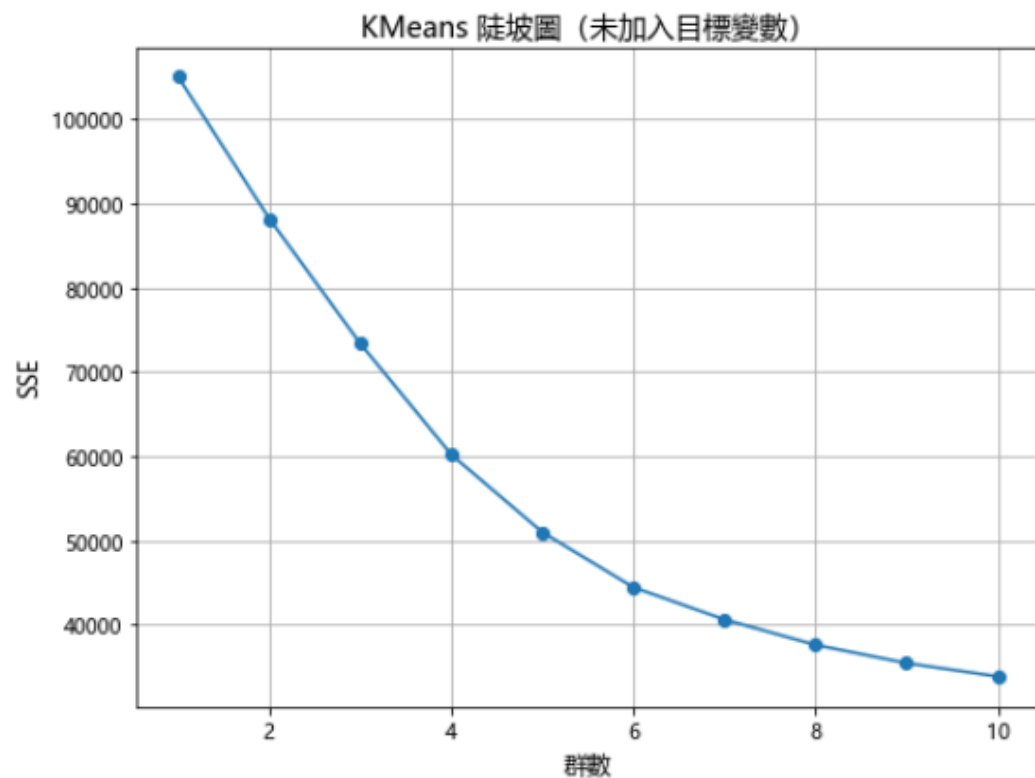
	last_evaluation	number_project	...	average_monthly_hours	satisfaction_level
0	0.670777	0.629113	...	0.674942	-0.132580
1	-0.114407	-0.058446	...	-0.046953	0.136253
2	-0.491687	-0.462822	...	-0.497077	0.093287

Python Console History

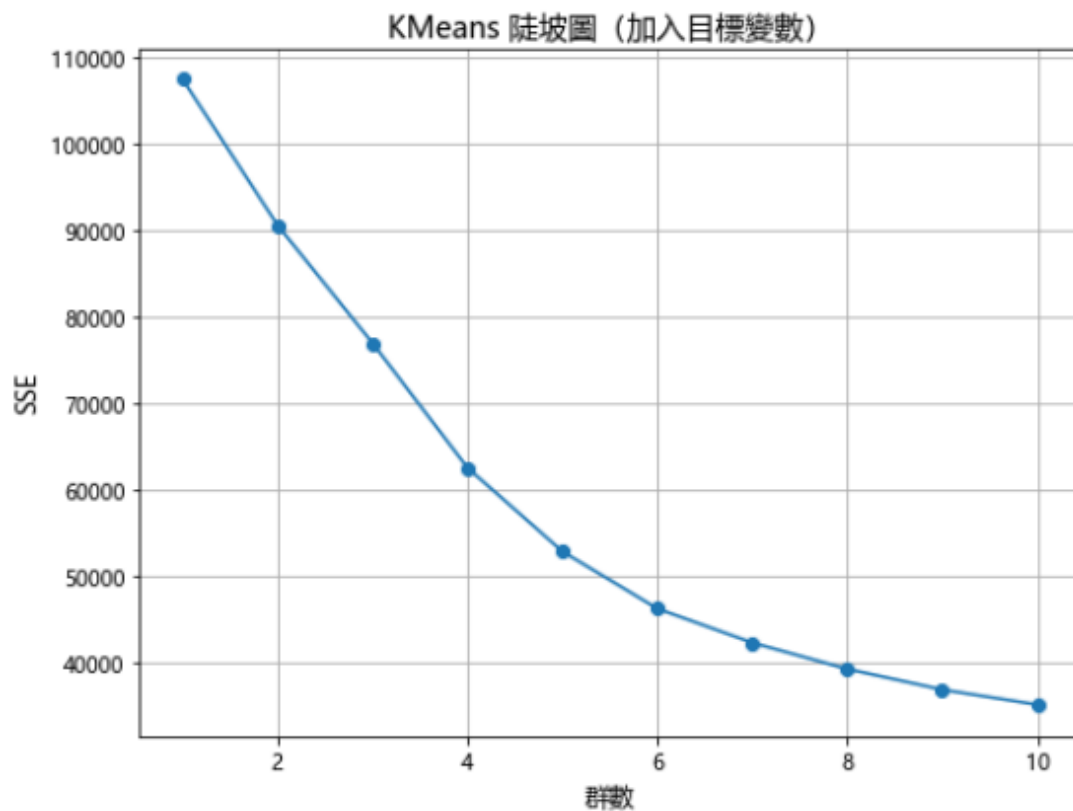
conda (Python 3.11.5) Completions: conda LSP: Python Line 60, Col 87 UTF-8 CRLF RW Mem 39%

K-Means

K-Means 陡坡圖



未加入目標變數的情況下，最佳群數可以選擇 **K=4**，因為此時 SSE 下降趨勢顯著減緩。



加入目標變數後，數據的分群效果可能更接近業務需求，推薦選擇 **K=5** 作為最佳群數

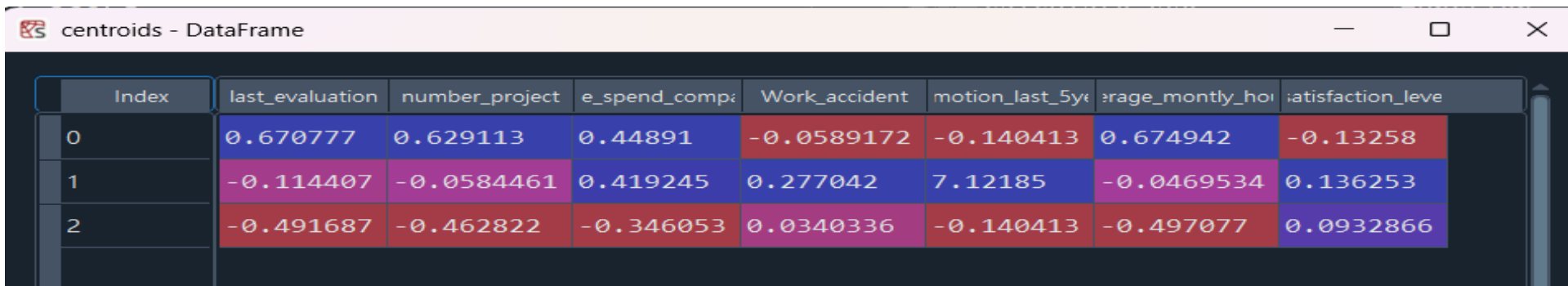
K-Means

K-Means_比較SSE

條件	分群數	SSE	測試正確率
未加入目標變數	3	73339.814172	30%
加入目標變數	4	62573.690785	64%

K-Means

分群並說明各類別輪廓，並給予各輪廓的行銷命名



Index	last_evaluation	number_project	e_spend_comp	Work_accident	motion_last_5y	average_monthly_hours	satisfaction_level
0	0.670777	0.629113	0.44891	-0.0589172	-0.140413	0.674942	-0.13258
1	-0.114407	-0.0584461	0.419245	0.277042	7.12185	-0.0469534	0.136253
2	-0.491687	-0.462822	-0.346053	0.0340336	-0.140413	-0.497077	0.0932866

群編號	行銷命名	特徵描述	代表性比例
0	工作狂專業族	擁有較高的 last_evaluation、number_project 和 average_monthly_hours，表現出高生產力與高壓力指標，滿意度略低	67%
1	穩定持續型員工	平均的 last_evaluation 與中等的滿意度，number_project 和 average_monthly_hours 均接近中間值	50%
2	低投入族群	last_evaluation、number_project 和 average_monthly_hours 顯著低，滿意度較高，可能工作投入度不足	32%

總結

通過這些模型的結果，可以看出 **satisfaction_level**（滿意度）、**time_spend_company**（在公司工作的時間）、**number_project**（參與的專案項目數）、**average_monthly_hours**（每月平均工作時間）和 **promotion_last_5years**（過去五年內是否有晉升）等變數對員工離職率有顯著影響。這些變數的重要性在不同模型中有所體現，並提供了不同的視角來理解員工離職的原因。根據這些結果，可以制定針對性的策略來減少員工流失率。

總結

應用建議：

1.離職原因洞察：

1. 工作負擔（項目數與時數）適中但壓力大，是離職的重要驅動因素。
2. 工齡短、滿意度高的員工對未來缺乏安全感，離職率更高。

2.留任原因分析：

1. 項目負擔輕、壓力較小的員工往往選擇穩定留任。
2. 收入穩定或績效評估較低的員工反而更傾向於保持現狀。

3.HR 行動建議：

1. 對滿意度高但工齡短的員工提供更多發展機會和支持，減少離職率。
2. 針對項目數量適中的員工，定期提供壓力管理方案。
3. 對薪資普通或績效較低但留任意願高的員工，給予適當激勵，維持穩定性。



THANK YOU!